EYouth
Data Analytics
Bootcamp

E-commerce Data Analysis Documentation

Team “13”

Mohamed Salama Abdelaziz

Rawda Mahmoud Yehya

Eman Gamal Badry

Nada Ahmed Mohamed

Ahmed Essam

Mohamed Ali

Table of Contents

Introduction:

As e-commerce continues to expand, leveraging data analytics has become essential for businesses to remain competitive and customer-focused. This project aims to explore and analyse e-commerce data through three key objectives:

1. **Sales Forecasting** – to predict future sales trends and support inventory and marketing planning.
2. **Product Analysis** – to identify high-performing and underperforming products, helping guide strategic decisions regarding promotions, pricing, and stock management.
3. **Customer Segmentation** – to classify customers into distinct groups based on behaviour and demographics, enabling more personalized marketing and improved customer retention.

The analysis was conducted using tools such as SQL for data extraction, Python for advanced analytics and modeling, and Power BI for interactive dashboards and visual storytelling. The process followed a comprehensive data pipeline including data cleaning, exploration, modeling, and visualization — all designed to deliver actionable insights that support data-driven decision-making in the e-commerce domain.

Azure SQL Database Setup & Power BI Integration

1. Environment Overview

- **Project Purpose**: Host an SQL database in the cloud to enable data access, user management, and real-time integration with Power BI dashboards for data visualization and analysis.
- **Cloud Provider**: Microsoft Azure
- **Subscription Type:** Azure for Students (Free Tier)

---

2. Azure SQL Server Configuration

- **Server Name:** server-25
- **Admin Username:** MohamedAli
- **Region:** UAE North
- **Authentication:** SQL Authentication
- **Elastic Pool:** Not used (dedicated resources)
- **Connection Strings**: Managed via Azure portal (SQL Auth)
- **Restore Point:** First available restore point: May 18, 2025, 21:51 UTC

---

3. Database Setup

- **Database Name:** Database-ecommerce_db
- **Resource Group:** Database-ecommerce_db
- **Status: Online**
- **Pricing Tier:** General Purpose: Gen5, 2 vCores
    - Covered under Free Tier benefits of *Azure for Students*

    - Monitoring performed to ensure usage stays within free quota
- **Storage:** Standard (default setup under student subscription)

---

4. Firewall Configuration

To enable external users to connect, IP addresses were whitelisted in the SQL Server firewall settings.

User	IP Address
Rawda	156.199.155.193
Mohamed Salama	41.199.23.253
Vanna	34.123.130.126
Mohamed (local)	41.33.88.71

All IPs were added using:

- **Start IP** = **End IP** (for single IP access)

---

5. User Management & Permissions

- Connected to the server using **SQL Server Management Studio (SSMS)** with:
    - **Server**: server-25.database.windows.net
    - **Authentication**: SQL Authentication
    - **Database**: Database-ecommerce_db

Created users with read-only permissions:

Admin-level changes were made only via the MohamedAli account.

---

6. Power BI Integration

Steps to connect Power BI to the Azure SQL Database:

1. Opened **Power BI Desktop**
2. Selected **Get Data > SQL Server**
3. Entered:
    - **Server**: server-25.database.windows.net
    - **Database**: Database-ecommerce_db
    - **Authentication Mode**: SQL Authentication
4. Selected **DirectQuery** mode to enable real-time queries
5. Loaded tables and created visual dashboards showing:
    - Sales volume
    - Top-selling products
    - Performance by region
    - Monthly KPIs

---

7. Cost Monitoring

Although this setup used **General Purpose: Gen5, 2 vCores**, **Azure for Students** covers these resources. No additional charges occurred, confirmed via:

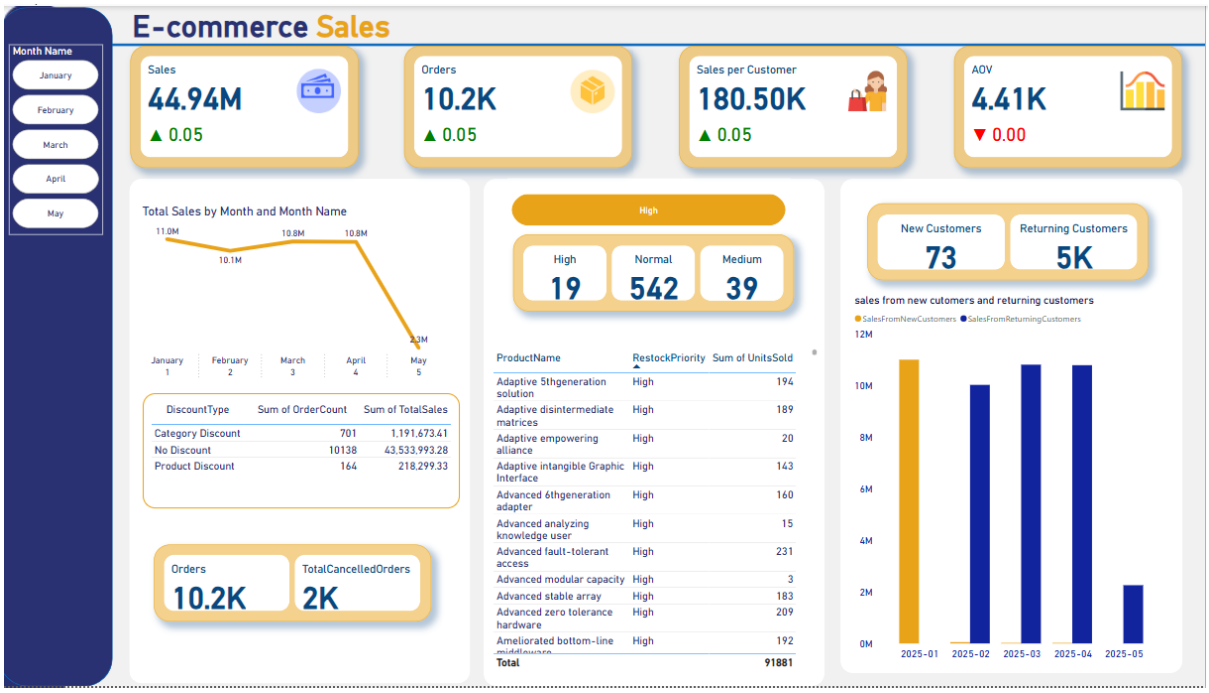
- **Azure Cost Management + Billing**
- Usage consistently monitored to avoid exceeding free quota.

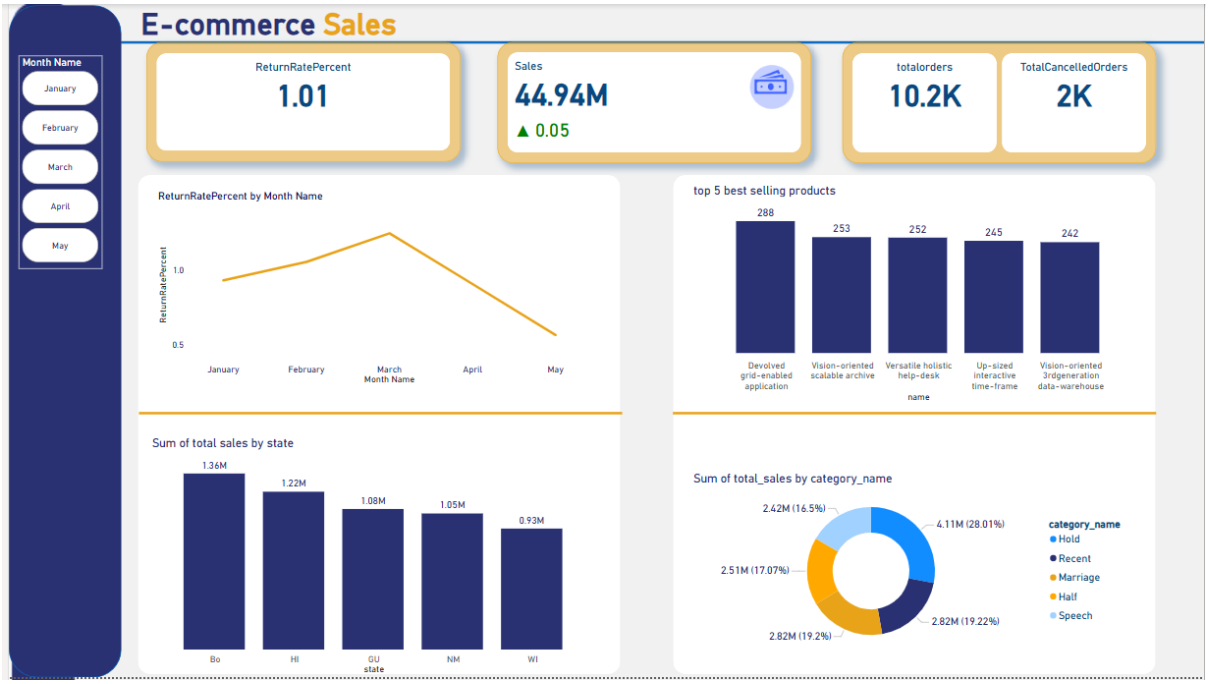
Summary of Completed Tasks

Task	Status
Created Azure SQL Server	✅
Created SQL Database	✅
Configured Firewall IPs	✅

Task	Status
Created SQL Users with Permissions	☑
Connected Database to Power BI	☑
Monitored Usage & Cost	☑

Power Bi Dashboard(Sales page)





Overall Impression:

The dashboard provides a good overview of key e-commerce metrics, allowing for quick insights into sales performance, order volume, customer value, and product performance. The use of various chart types (line, bar, donut) effectively visualizes different aspects of the data.

Key Observations and Inferences:

1. **Sales and Order Performance:**
    - **Total Sales:** $44.94M, showing a slight increase (0.05% up).
    - **Total Orders:** 10K, also with a slight increase (0.05% up).
    - **Sales per Customer:** $179.78K, with a slight increase (0.05% up).
    - **AOV (Average Order Value):** $4.41K, showing no change (0.00% change).
    - **Inference:** While there's a slight positive trend in sales, orders, and sales per customer, the flat AOV suggests that the growth might be driven by more transactions or customers, rather than customers spending more per order.
2. **Sales Trend (Monthly):**
    - The "Total Sales by Month Name" line chart shows a clear downward trend from January to May. Sales peaked in January and have been consistently declining.
    - **Inference:** This is a critical area for concern. The business needs to investigate the reasons for this sales decline. Possible factors include seasonality, increased competition, marketing issues, or product availability.

3. **Sales by Category:**
    - The donut chart "Sum of total_sales by category_name" highlights the top-performing categories. "Hold" (2.82M, 19.22%), "Recent" (2.51M, 17.07%), and "Marriage" (2.42M, 16.5%) are the largest contributors to total sales.
    - **Inference:** Understanding the top categories can help in inventory management, targeted marketing, and promotional strategies. Categories like "Half" and "Speech" contribute less and might need attention.
4. **Sales by State:**
    - The "Sum of total sales by state" bar chart indicates that "Bo" (presumably a state or region abbreviation) has the highest sales at $1.36M, followed by "Hi" at $1.22M. "Wl" has the lowest at $0.93M.
    - **Inference:** This data is valuable for regional marketing efforts, optimizing logistics, and potentially identifying underperforming regions that need more focus.
5. **Discount Impact:**
    - The "DiscountType" table shows that "No Discount" sales account for a significant portion ($1,191,673.41 total sales from 43,538 orders), compared to "Category Discount" ($701 from 701 orders) and "Product Discount" ($218,299.33 from 164 orders).
    - **Inference:** This suggests that a large portion of sales occurs without discounts, which is positive for-profit margins. However, it's also worth investigating if strategic discounting could boost sales further without cannibalizing full-price purchases. The very low number of "Category Discount" orders and sales might indicate it's not effectively utilized or targeted.
6. **Product Restock Priority:**
    - The "Product Name" table with "RestockPriority" is a crucial operational insight. Many products are categorized as "Medium" priority for restock, with "Sum of Units Sold" indicating quantities. For instance, "Vision-oriented well-modulated database" has 17 units sold and a "Medium" restock priority.
    - **Inference:** This table is essential for inventory management. Products with "High" restock priority should be addressed immediately to prevent stockouts and lost sales. "Medium" priority items need to be monitored closely

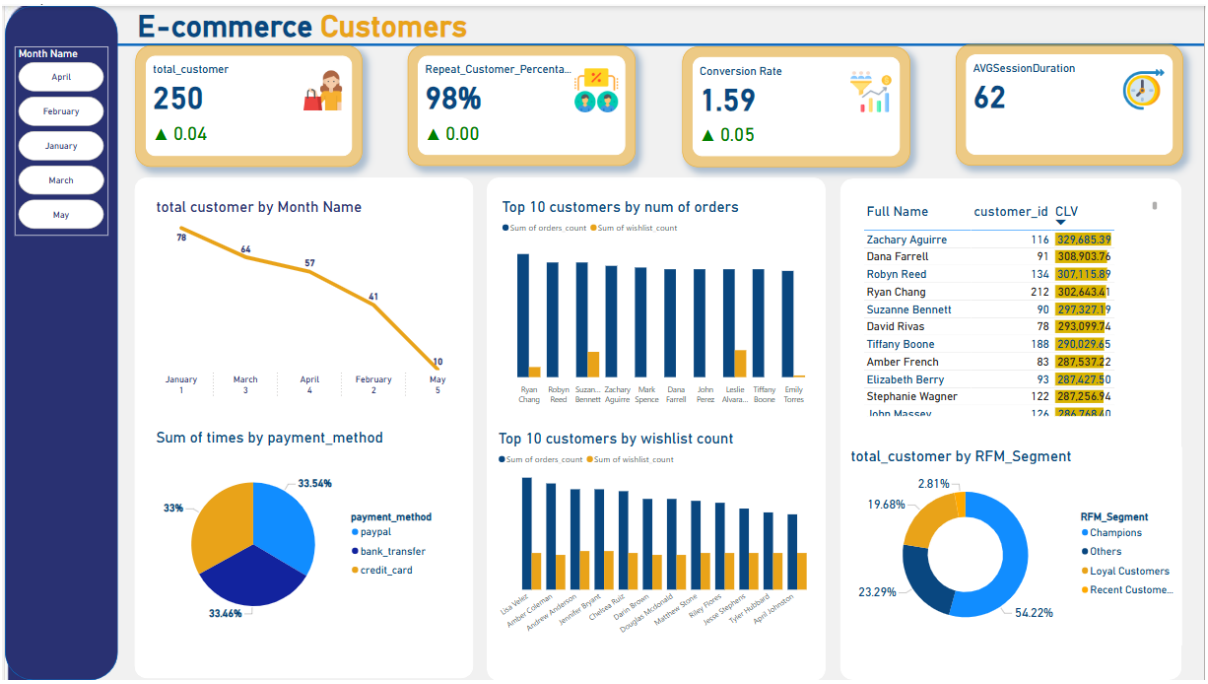
7. **Top 5 Best Selling Products:**

- The bar chart "top 5 best-selling products" shows the product names (which are cut off but appear to be "Devolved vision-enl... ", "Versatile head-del... ", "Up-sized vision-ori... ", etc.) and their respective sales values (288, 253, 252, 245, 242).
- **Inference:** Identifying the top-selling products allows the business to prioritize these items in terms of marketing, promotions, and inventory. It also offers insights into what customers are currently most interested in.

Recommendations/Next Steps:

- **Investigate Sales Decline:** Deep dive into the reasons for the consistent sales decline from January to May. This might involve looking at website traffic, conversion rates, marketing campaign performance, seasonal trends, and competitor activities.
- **Optimize Discounting Strategy:** Analyze the effectiveness of different discount types. Are "Category Discounts" being used effectively? Could strategic product-level discounts boost sales during slow periods without eroding margins too much?
- **Regional Focus:** Leverage the sales by state data to tailor marketing campaigns or promotions to specific regions, especially for underperforming states like "Wl."
- **Inventory Management:** Act on the "RestockPriority" data to ensure that high-priority items are restocked promptly to avoid missed sales opportunities.
- **Customer Behavior:** Further analyze sales per customer and AOV to understand if there are opportunities to increase customer spending (e.g., through cross-selling or up-selling).
- **Product Performance:** Use the top 5 selling products data to inform future product development, marketing efforts, and inventory planning.

Power Bi Dashboard(Customers page)



Overall Impression:

This dashboard effectively focuses on customer-centric metrics, offering insights into customer acquisition, retention, behavior, and value. It complements the previous sales dashboard by shedding light on the "who" behind the transactions.

Key Observations and Inferences:

1. **Key Customer Metrics:**
    - **Total Customer:** 250, with a slight increase (0.04% up). This indicates a slow but positive growth in the customer base.
    - **Repeat Customer Percentage:** 98%, with no change (0.00%). This is an exceptionally high percentage and suggests strong customer loyalty and retention. This is a significant strength for the business.
    - **Conversion Rate:** 1.59%, with a slight increase (0.05% up). While showing positive movement, 1.59% is generally on the lower side for e-commerce and indicates room for improvement in converting visitors into customers.
    - **AVG Session Duration:** 62 (units not specified, but likely seconds). This metric provides an idea of how engaged users are on the site.
2. **Customer Acquisition Trend (Monthly):**
    - The "total customer by Month Name" line chart shows a similar downward trend to sales: 78 new customers in January, dropping significantly to 10 in May.

- **Inference:** This mirrors the sales decline observed in the previous dashboard. The business is acquiring fewer new customers each month, which, despite the high repeat customer rate, will eventually impact overall sales if not addressed.

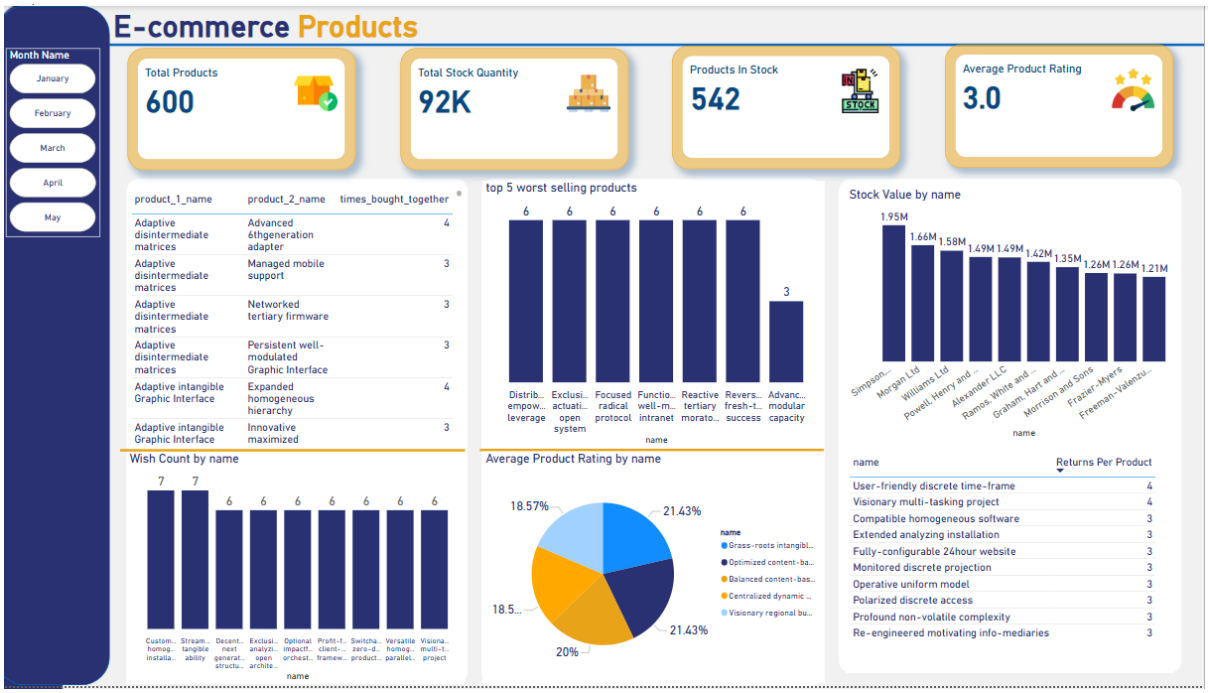
3. **Payment Method Preference:**
   - The "Sum of times by payment_method" donut chart shows that "credit_card" (33.66%) and "bank_transfer" (33.54%) are almost equally preferred, followed closely by "paypal" (32.81%).
   - **Inference:** This indicates a balanced preference across major payment methods. Ensuring seamless and secure options for all these methods is crucial. There isn't a dominant preferred method, which might suggest customer flexibility or that all options are equally viable for different customer segments.
4. **Top 10 Customers by Orders and Wishlist:**
   - The "Top 10 customers by num of orders" and "Top 10 customers by wishlist count" bar charts identify high-value and engaged customers. For example, "Ryan" (likely Ryan Chang from the table) is prominent in both, indicating he's a frequent buyer and also actively uses the wishlist.
   - **Inference:** These lists are invaluable for identifying "VIP" customers. Businesses can leverage this information for personalized marketing, loyalty programs, or special offers to reward and further engage these top customers. Customers with high wishlist counts but lower order counts might need targeted promotions to convert their interest into purchases.
5. **Customer Lifetime Value (CLV):**
   - The "Full Name" table lists customer IDs, their number of orders, and their CLV. For example, "Zachary Aguirre" has a CLV of $329,485.39, indicating he is a very high-value customer. "Diane Farrell" and "Robyn Reed" also show high CLVs.
   - **Inference:** This table is critical for understanding which customers generate the most revenue over time. Businesses should focus retention efforts on these high-CLV customers and analyze their behavior to replicate their success with other customer segments.
6. **Customer Segmentation (RFM):**
   - The "total_customer by RFM_Segment" donut chart breaks down customers into segments based on Recency, Frequency, and Monetary value. "Champions" (54.62%) make up over half of the customer base, followed by "Loyal Customers" (23.29%). "Recent Customers" (19.28%) are also a significant segment, while "Others" (2.81%) represent a smaller group.
   - **Inference:** The very high percentage of "Champions" and "Loyal Customers" reinforces the strong repeat customer rate observed

earlier. This is a tremendous asset. The business has a highly engaged and valuable customer base. Marketing efforts should focus on retaining these segments and nurturing "Recent Customers" to move into higher RFM segments. The "Others" segment might need re-engagement strategies or could represent fewer valuable customers.

Recommendations/Next Steps:

- **Address Customer Acquisition Decline:** Investigate the sharp drop in new customer acquisition from January to May. This likely correlates with the sales decline and requires immediate attention to marketing channels, SEO, advertising spend, and acquisition funnels.
- **Leverage High Repeat Customer Rate:** Capitalize on the 98% repeat customer rate by implementing robust loyalty programs, exclusive offers for existing customers, and personalized communication to further strengthen retention.
- **Improve Conversion Rate:** Analyze the user journey and website experience to identify bottlenecks contributing to the relatively low conversion rate. A/B testing, clearer calls to action, simplified checkout processes, and improved product descriptions could help.
- **Deep Dive into Top Customers:** Conduct a more in-depth analysis of "Champions" and top CLV customers. What common characteristics do they share? How can their behaviours be encouraged in other segments?
- **Personalize Marketing:** Use the insights from top customers (by orders, wishlist, and CLV) to create highly personalized marketing campaigns. For example, target customers with high wishlist counts but low order counts with specific discounts on their wish-listed items.
- **Monitor RFM Segments:** Regularly track the movement of customers between RFM segments. The goal should be to move "Recent Customers" towards "Loyal Customers" and "Champions," and to re-engage or sunset the "Others" segment as appropriate.

Power Bi Dashboard(Products page)



Overall Impression:

This dashboard provides a comprehensive view of the product catalog, inventory status, and product performance. It's crucial for understanding what's being sold, what's in stock, and how products are being received by customers.

Key Observations and Inferences:

1. **Key Product Metrics:**
    - **Total Products:** 600. This is the total number of unique products in the catalog.
    - **Total Stock Quantity:** 92K. This represents the total units of all products currently in inventory.
    - **Products In Stock:** 542. This indicates that 542 out of 600 products (approximately 90.3%) currently have inventory.
    - **Average Product Rating:** 3.0 (out of an implied 5-star scale). This is a moderate average rating, suggesting that while products are generally satisfactory, there's significant room for improvement to reach higher customer satisfaction.
2. **Product List (Partial View):**
    - The table displaying `product_1_name` and `product_2_name` shows a list of product pairings or possibly parent/child product relationships. Without more context, it's hard to draw definitive conclusions, but it might indicate related products or different versions of the same product. For example, "Adaptive

5thgeneration solution" is paired with "Automated web-enabled collaboration."
    - **Inference:** This table is useful for understanding the product catalog structure and potentially for cross-selling strategies if `product_2_name` represents complementary items.
3. **Top 5 Worst Selling Products:**
    - The bar chart shows several products with very low sales (all at 6 units sold, except for one at 3 units). The product names are generic like "Disribut. Exclus. ...".
    - **Inference:** These are products that are not moving well. The business should investigate why they are performing poorly. Reasons could include low demand, poor marketing, high pricing, or inadequate product descriptions. Actions could involve discontinuing them, running promotions, or re-evaluating their position.
4. **Stock Value by Name:**
    - The bar chart displays the stock value for various products, with "Simpson..." at the highest with $1.95M, followed by "Margareth L.Ltd" at $1.66M, and others decreasing.
    - **Inference:** This highlights where the bulk of the inventory's monetary value lies. High stock value in a product indicates a significant investment. This data is critical for capital management, insurance, and understanding potential carrying costs. Products with high stock value but low sales might indicate overstocking issues.
5. **Wishlist Count by Name:**
    - The bar chart shows product names with their respective wishlist counts. Several products have 7 wishlist counts, and many others have 6.
    - **Inference:** Products on wishlists indicate customer interest, even if a purchase hasn't been made yet. Products with high wishlist counts but low sales could be opportunities for targeted promotions (e.g., "items on your wishlist are now X% off") to convert interest into sales.
6. **Average Product Rating by Name:**
    - The donut chart shows the distribution of average product ratings by various product names. The largest segments are 21.43%, 20%, 18.57%, and 21.43% (again), representing different named products.
    - **Inference:** This provides a breakdown of how well different products are being received. Products with lower average ratings (not explicitly shown here, but inferred from the average of 3.0) should be investigated. Customer feedback (reviews) should be

analyzed to understand reasons for low ratings and identify areas for product improvement or better customer expectation management.

7. **Returns Per Product:**
    - The table "Returns Per Product" lists product names and the number of returns for each. "User-friendly discrete time-frame" and "Visionary multi-tasking project" both have 4 returns, while others have 3 returns.
    - **Inference:** High return rates for specific products are a red flag. It could indicate issues with product quality, misrepresentation in product descriptions, or customer dissatisfaction. Investigating the reasons behind these returns is crucial to reduce costs and improve customer satisfaction.

Recommendations/Next Steps:

- **Improve Average Product Rating:** Identify products with ratings below the 3.0 average. Gather detailed customer feedback (reviews, surveys) to understand specific pain points and implement improvements where possible.
- **Address Worst-Selling Products:** For the products with the lowest sales, determine the root cause of their poor performance. Consider:
    - Marketing campaigns to increase visibility.
    - Price adjustments.
    - Bundle offers.
    - If no improvement, consider discontinuing them to free up inventory space and capital.
- **Optimize Inventory for High Stock Value:** Monitor products with high stock value closely. If sales are slow for these items, consider promotions to move inventory, or adjust future ordering to prevent overstocking.
- **Convert Wishlist Items:** Implement strategies to convert wishlist items into purchases, such as automated emails when wishlisted items go on sale or are back in stock.
- **Reduce Returns:** For products with high return rates, analyze the reasons for returns (e.g., defective, not as described, wrong size). Address these issues through product quality control, improved descriptions, or better sizing guides.
- **Analyze Product Pairings:** If `product_1_name` and `product_2_name` represent bundles or complementary items, analyze their combined sales performance and explore opportunities for cross-selling or creating attractive packages.

SQL queries

```sql
--1 Calculate the total sales revenue from all orders.

SELECT
    SUM(od.quantity * od.unit_price) AS total_sales_revenue
FROM
    order_details od
JOIN
    Orders o ON od.order_id = o.id
WHERE
    o.status IN ('processing', 'shipped', 'delivered');
```

Results Messages

	total_sales_revenue
1	26768217.74

Query executed successfully.

```sql
--2 List the top 5 best-selling products by quantity sold.
SELECT TOP 5
       p.id,p.name,SUM(od.quantity) AS Total_Quantity
       FROM order_details od
       JOIN products p
       on od.product_id=p.id
       GROUP BY p.id, p.name
       ORDER BY Total_Quantity DESC
```

Results Messages

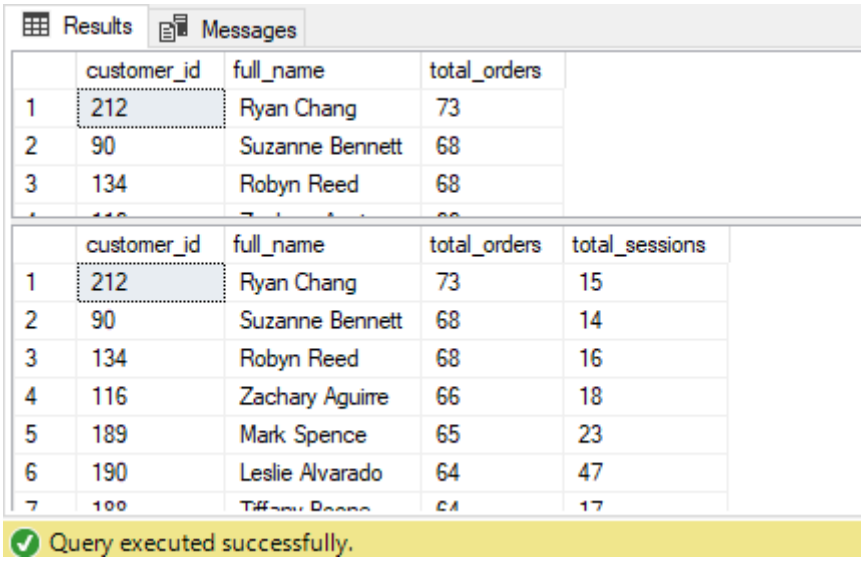
	id	name	Total_Quantity
1	591	Vision-oriented scalable archive	253
2	499	Versatile holistic help-desk	252
3	216	Up-sized interactive time-frame	245
4	195	Vision-oriented 3rdgeneration data-warehouse	242
5	348	Team-oriented discrete hierarchy	241

Query executed successfully.

```sql
--3 Identify customers with the highest number of orders + num of support sessions
 SELECT
```

```
    c.id AS customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    COUNT(o.id) AS total_orders
FROM customers c
JOIN orders o ON c.id = o.customer_id
GROUP BY c.id, c.first_name, c.last_name
ORDER BY total_orders DESC;

WITH orders_count AS (
    SELECT customer_id, COUNT(*) AS total_orders
    FROM orders
    GROUP BY customer_id
),
sessions_count AS (
    SELECT customer_id, COUNT(*) AS total_sessions
    FROM customer_sessions
    GROUP BY customer_id
)
SELECT
    c.id AS customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    ISNULL(o.total_orders, 0) AS total_orders,
    ISNULL(s.total_sessions, 0) AS total_sessions
FROM customers c
LEFT JOIN orders_count o ON c.id = o.customer_id
LEFT JOIN sessions_count s ON c.id = s.customer_id
ORDER BY total_orders DESC;
```



Results | Messages

	customer_id	full_name	total_orders
1	212	Ryan Chang	73
2	90	Suzanne Bennett	68
3	134	Robyn Reed	68

	customer_id	full_name	total_orders	total_sessions
1	212	Ryan Chang	73	15
2	90	Suzanne Bennett	68	14
3	134	Robyn Reed	68	16
4	116	Zachary Aguirre	66	18
5	189	Mark Spence	65	23
6	190	Leslie Alvarado	64	47

Query executed successfully.

```
--4 Generate an alert for products with stock quantities below 20 units.--> to full it
again

SELECT
    p.id AS product_id,
    p.name AS product_name,
    i.quantity
FROM products p
JOIN inventory_movements i ON p.id = i.product_id
WHERE i.quantity < 20;
```

Results | Messages

	product_id	product_name	quantity
1	38	Self-enabling leadingedge utilization	-3
2	35	Inverse holistic open architecture	-5
3	46	Exclusive dedicated middleware	-3
4	36	Total 24hour synergy	-1
5	83	Grass-roots 4thgeneration open system	-5
6	29	Synchronized multi-tasking hierarchy	-5
7	48	Compatible modular policy	-1
8	13	Horizontal didactic standardization	-4
9	32	Re-contextualized 24hour Local Area Network	-5
10	38	Self-enabling leadingedge utilization	-3
11	53	Realigned discrete Internet solution	-2

Query executed successfully.

```sql
--5 Determine the percentage of orders that used a discount.

SELECT
    ROUND(
        COUNT(DISTINCT od.order_id) * 100.0 /
        (SELECT COUNT(*) FROM orders),
        2
    ) AS percentage_of_discounted_orders
FROM
    order_details od
INNER JOIN
    discounts d
       ON od.product_id = d.product_id
WHERE
    d.is_active = 1;
```

	percentage_of_discounted_orders
1	6.250000000000

Query executed successfully.

```sql
--6 Calculate the average rating for each product.

select  p.name as Product , AVG(rating) as AVG_Rating
from reviews r
inner join products p
on p.id = r.product_id
```

```
group by p.name;
```

	Product	AVG_Rating
1	Adaptive 5thgeneration solution	3
2	Adaptive disintermediate matrices	2
3	Adaptive empowering alliance	2
4	Adaptive intangible Graphic Interface	2
5	Advanced 6thgeneration adapter	3
6	Advanced analyzing knowledge user	3
7	Advanced fault-tolerant access	2
8	Advanced modular capacity	2
9	Advanced stable array	3
10	Advanced zero tolerance hardware	2
11	Ameliorated bottom-line middleware	3

Query executed successfully.

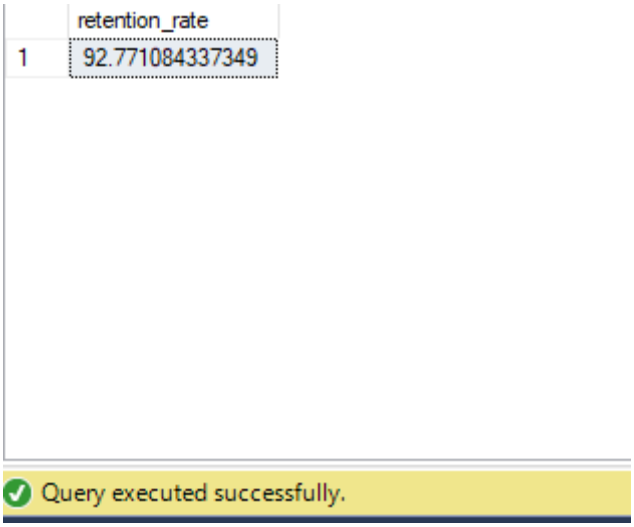
```
--------------------------------------------------------------------------------
---- Advanced Queries

--1 Compute the 30-day customer retention rate after their first purchase

WITH FirstPurchase AS(
SELECT
	customer_id,
	MIN(order_date) AS first_purchase_date
FROM
	orders
GROUP BY
		customer_id
),

PurchasesAfter30Days AS(
	SELECT
		fp.customer_id
	FROM
		FirstPurchase fp
	join
		orders o
	on  fp.customer_id = o.customer_id
	WHERE
	o.order_date>fp.first_purchase_date
	AND o.order_date<=DATEADD(day, 30, fp.first_purchase_date)

)
SELECT
    (COUNT(DISTINCT pa.customer_id) * 100.0 / COUNT(DISTINCT fp.customer_id)) AS
retention_rate
FROM
    FirstPurchase fp
LEFT JOIN
    PurchasesAfter30Days pa ON fp.customer_id = pa.customer_id;
```

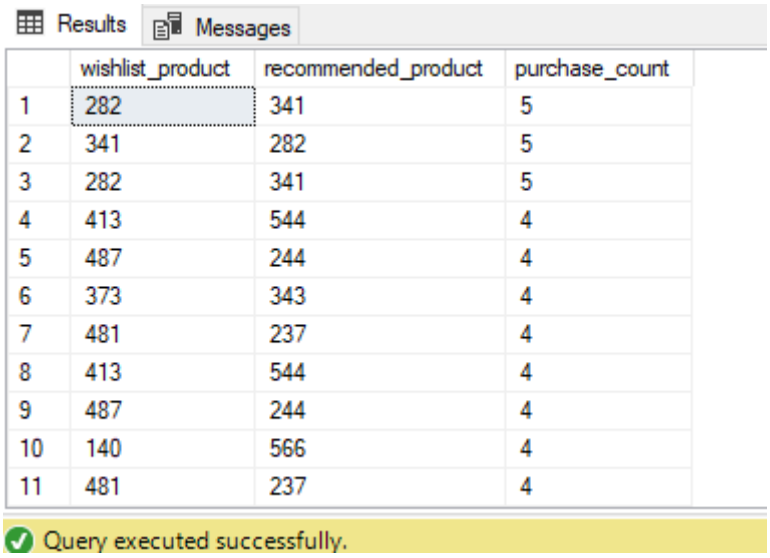



```sql
--2 Recommend products frequently bought together with items in customer wishlists.

WITH WishlistItems AS (
    SELECT
        wl.customer_id,
        wl.product_id
    FROM
        Wishlists wl
    GROUP BY
        wl.customer_id, wl.product_id
),

FrequentlyBoughtTogether AS (
    SELECT
        od.product_id AS main_product,
        od2.product_id AS recommended_product,
        COUNT(*) AS purchase_count
    FROM
        order_details od
    JOIN
       order_details od2 ON od.order_id = od2.order_id
    WHERE
        od.product_id <> od2.product_id
    GROUP BY
        od.product_id, od2.product_id
)
SELECT
    wi.product_id AS wishlist_product,
    fbt.recommended_product,
    fbt.purchase_count
FROM
    WishlistItems wi
JOIN
    FrequentlyBoughtTogether fbt ON wi.product_id = fbt.main_product
ORDER BY

   fbt.purchase_count DESC;
```



Results | Messages

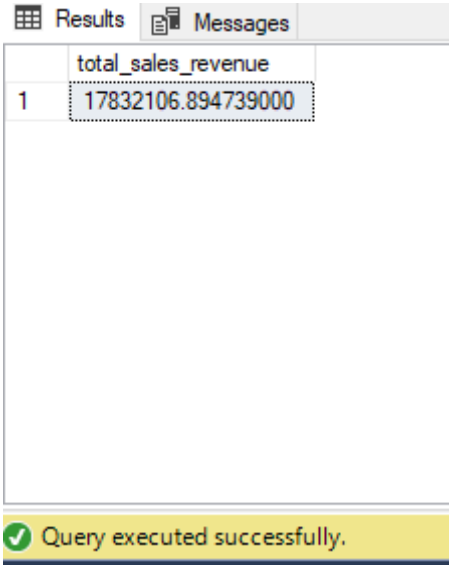
	wishlist_product	recommended_product	purchase_count
1	282	341	5
2	341	282	5
3	282	341	5
4	413	544	4
5	487	244	4
6	373	343	4
7	481	237	4
8	413	544	4
9	487	244	4
10	140	566	4
11	481	237	4

Query executed successfully.

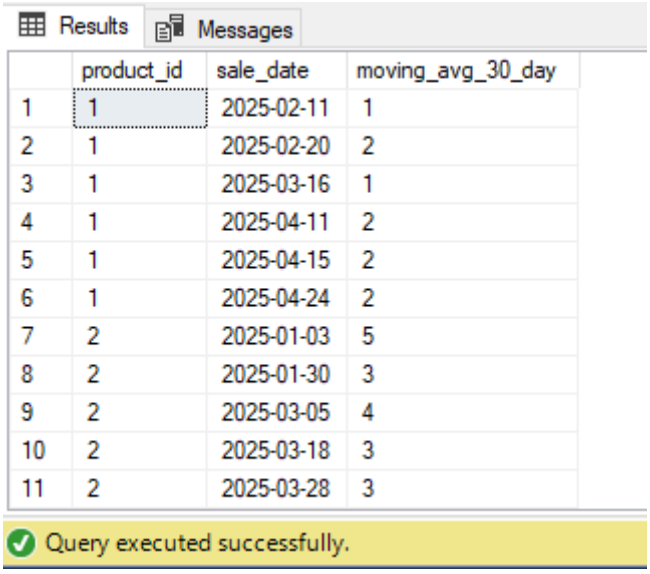
```sql
--
WITH RankedDiscounts AS (
    SELECT
        d.id AS discount_id,
        d.percentage,
        d.product_id AS d_product_id,
        d.category_id,
        d.start_date,
        d.end_date,
        d.is_active,
        od.order_id,
        od.product_id AS od_product_id,
        ROW_NUMBER() OVER (
            PARTITION BY od.order_id, od.product_id
            ORDER BY
                CASE
                    WHEN d.product_id IS NOT NULL THEN 1
                    ELSE 2
                END,
                d.percentage DESC
        ) AS rn
    FROM order_details od
    JOIN Orders o ON od.order_id = o.id
    LEFT JOIN Discounts d ON (
        (d.product_id IS NULL OR d.product_id = od.product_id)
        AND (d.category_id IS NULL OR d.category_id = (
            SELECT category_id FROM Products WHERE id = od.product_id
        ))
        AND d.start_date <= o.order_date
        AND d.end_date >= o.order_date
        AND d.is_active = 1
    )
    WHERE o.status IN ('processing', 'shipped', 'delivered')
)

SELECT
    SUM(od.quantity * od.unit_price * (1 - COALESCE(rd.percentage, 0) / 100.0)) AS
total_sales_revenue
FROM
    order_details od
JOIN
    Orders o ON od.order_id = o.id
```

```
LEFT JOIN
    RankedDiscounts rd ON rd.order_id = od.order_id
                      AND rd.od_product_id = od.product_id
                      AND rd.rn = 1
LEFT JOIN
    Returns r ON r.order_id = o.id AND r.status = 'approved'
WHERE
    o.status IN ('processing', 'shipped', 'delivered')
    AND r.order_id IS NULL;
```




```
--3 Track inventory turnover trends using a 30-day moving average.
WITH sales_per_day AS (
    SELECT
        oi.product_id,
        CAST(o.order_date AS DATE) AS sale_date,
        SUM(oi.quantity) AS daily_quantity
    FROM order_details oi
    JOIN orders o ON oi.order_id = o.id
    GROUP BY oi.product_id, CAST(o.order_date AS DATE)
),
moving_avg AS (
    SELECT
        product_id,
        sale_date,
        AVG(daily_quantity) OVER (
            PARTITION BY product_id
            ORDER BY sale_date
            ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
        ) AS moving_avg_30_day
    FROM sales_per_day
)
SELECT *
FROM moving_avg
ORDER BY product_id, sale_date;
```



Results | Messages

	product_id	sale_date	moving_avg_30_day
1	1	2025-02-11	1
2	1	2025-02-20	2
3	1	2025-03-16	1
4	1	2025-04-11	2
5	1	2025-04-15	2
6	1	2025-04-24	2
7	2	2025-01-03	5
8	2	2025-01-30	3
9	2	2025-03-05	4
10	2	2025-03-18	3
11	2	2025-03-28	3

Query executed successfully.

```sql
--4 Identify customers who have purchased every product in a specific category.
WITH category_products AS (
    SELECT id AS product_id
    FROM products
          WHERE category_id = 10 -- Chose ur category
),
customer_purchases AS (
    SELECT DISTINCT o.customer_id, oi.product_id
    FROM orders o
    JOIN order_details oi ON o.id = oi.order_id
    WHERE oi.product_id IN (SELECT product_id FROM category_products)
),
customer_product_count AS (
    SELECT
        customer_id,
        COUNT(DISTINCT product_id) AS purchased_count
    FROM customer_purchases
    GROUP BY customer_id
),
total_category_products AS (
    SELECT COUNT(*) AS total_products
    FROM category_products
)
SELECT
    cpc.customer_id
FROM customer_product_count cpc
JOIN total_category_products tcp ON cpc.purchased_count = tcp.total_products;


--5  Find pairs of products commonly bought together in the same order.

SELECT
    od1.product_id AS product_1,
    od2.product_id AS product_2,
    COUNT(*) AS times_bought_together
FROM
    order_details od1
JOIN
    order_details od2
    ON od1.order_id = od2.order_id
    AND od1.product_id < od2.product_id
GROUP BY
```

```
    od1.product_id, od2.product_id
ORDER BY
    times_bought_together DESC;
```

Results | Messages

	product_1	product_2	times_bought_together
1	282	341	5
2	488	595	4
3	179	440	4
4	244	487	4
5	293	354	4
6	137	249	4
7	348	441	4
8	426	432	4
9	285	299	4
10	274	409	4
11	339	407	4

Query executed successfully.

```
--6  Calculate the time taken to deliver orders in days.

select o.id ,
DATEDIFF(day ,o.order_date,sh.shipping_date) as delivery_time
from orders o
inner join shipping sh
on o.id = sh.order_id

where sh.status = 'delivered'
```

Results | Messages

	id	delivery_time
1	8	1
2	9	1
3	17	1
4	33	1
5	34	1
6	36	1
7	38	1
8	39	1
9	44	1
10	45	1
11	47	1

Query executed successfully.

Python Dashboard

1 Introduction

This document provides an overview of the E-Commerce Dashboard, a web-based application built using Streamlit, SQL Server, Plotly, and Ollama. The dashboard is designed to provide actionable insights into sales, customer behavior, product performance, and interactive AI-driven analysis for an e-commerce platform. It includes four main pages: Sales, Customers, Products, and AI Chat, each offering metrics, visualizations, or query-based interactions.

2 Architecture

The application is structured as a multi-page Streamlit app with the following components:

• app.py: The main entry point, configuring the Streamlit app and navigation side bar.

• pages/: Directory containing individual page scripts:

– sales.py: Sales Dashboard page.

– customers.py: Customers Dashboard page.

– products.py: Products Dashboard page.

– ai_chat.py : AI Chat page.

• Dependencies:

– Python 3.12

– Streamlit: For the web interface.

– Pandas: For data manipulation.

– SQLAlchemy and pyodbc: For SQL Server connectivity.

– Plotly: For interactive visualizations.

– Ollama: For generating SQL queries from natural language.

3 Pages

3.1 Sales Dashboard

The Sales Dashboard provides an overview of sales performance with the following features:

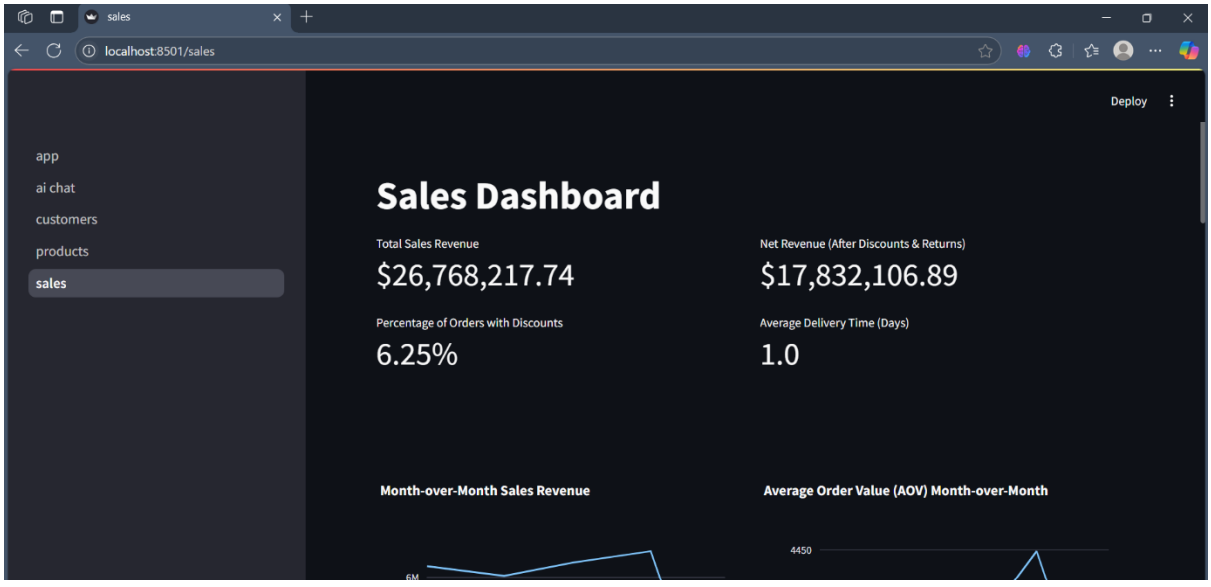
3.1.1 Metrics

• Total Sales Revenue: Total revenue from orders in ”processing”, ”shipped”, or ”delivered” status.

• Net Revenue (After Discounts & Returns): Revenue after applying discounts and excluding returns.

• Percentage of Orders with Discounts: Percentage of orders utilizing active discounts.

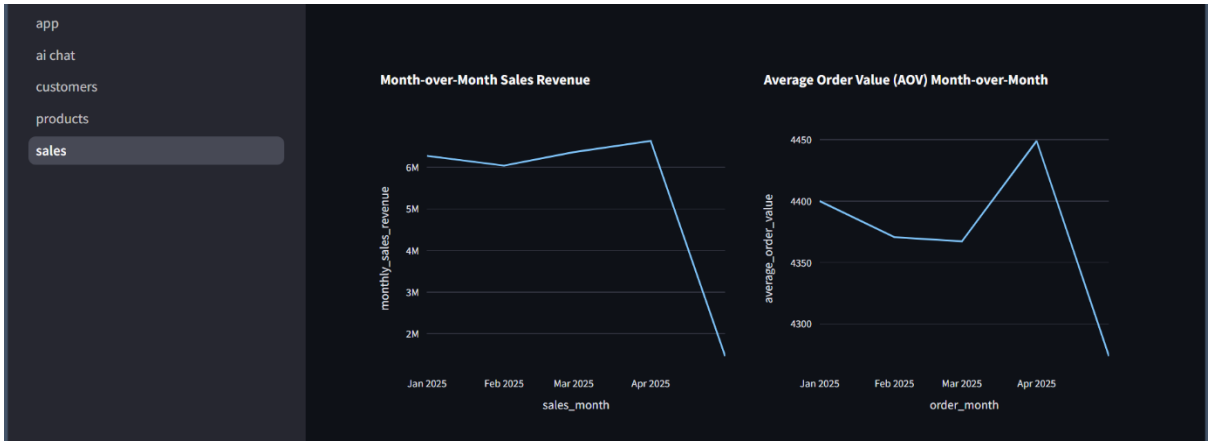
• Average Delivery Time (Days): Average time from order placement to delivery.



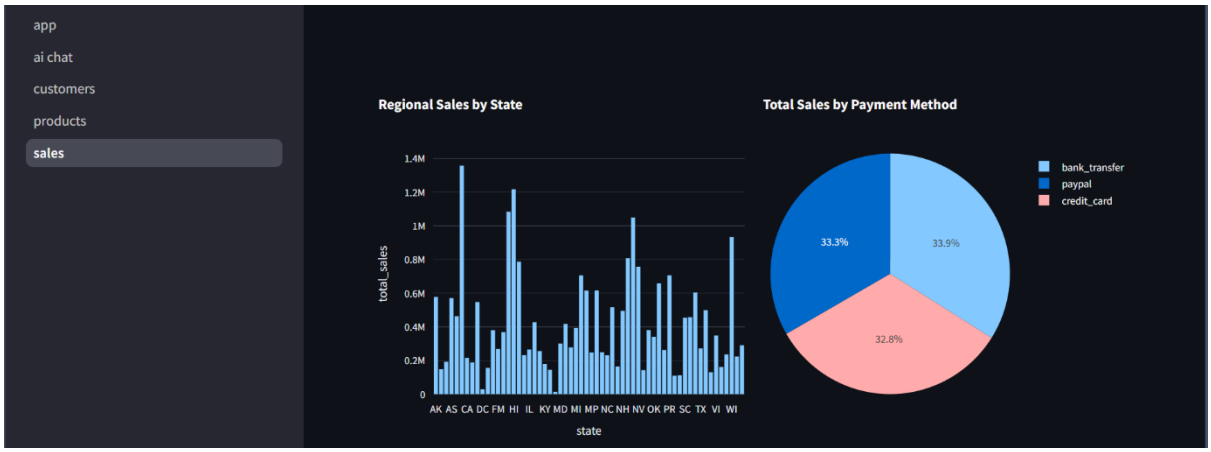
3.1.2 Charts

Charts are displayed in pairs with vertical spacing between rows:

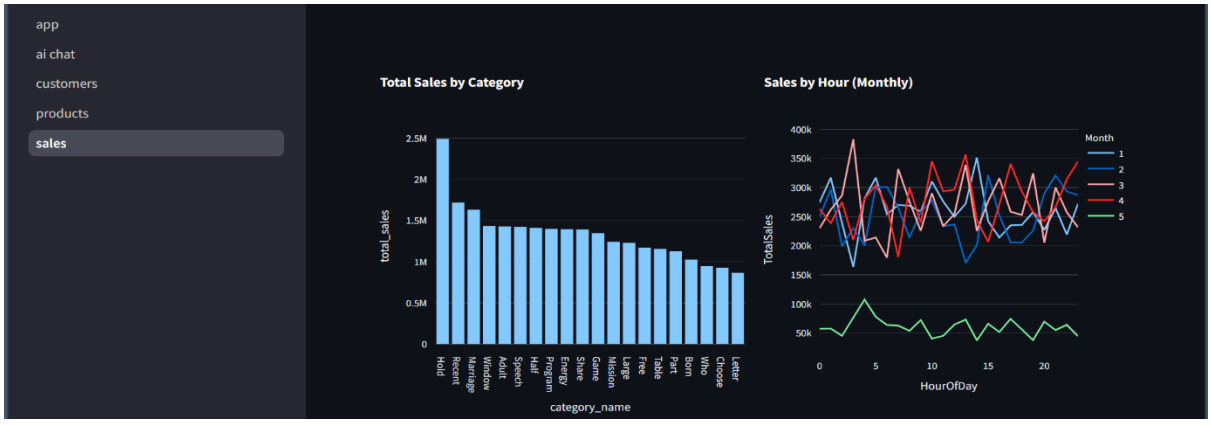
• Month-over-Month Sales Revenue and Average Order Value (AOV)Month over-Month: Line charts tracking sales and AOV trends.



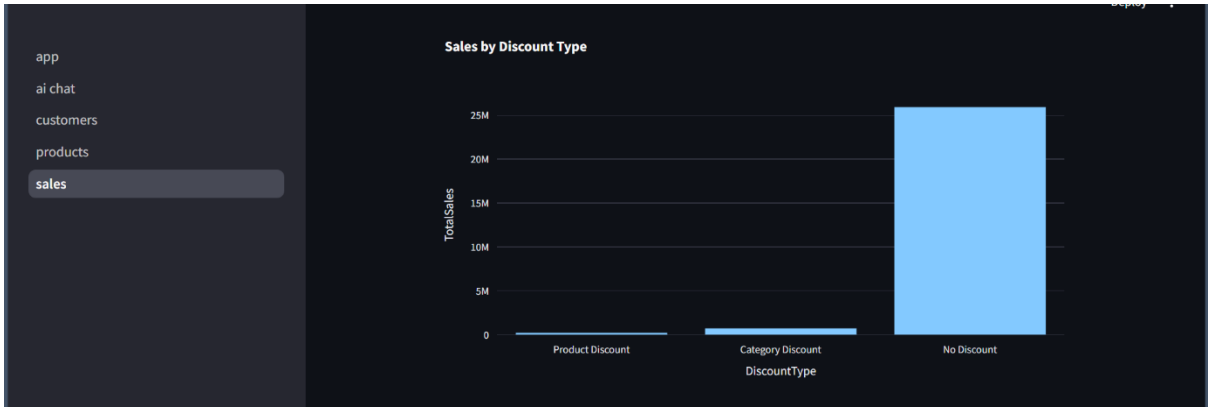
• Regional Sales by State and Total Sales by Payment Method: Bar chart for regional sales and pie chart for payment method distribution.



• Total Sales by Category and Sales by Hour (Monthly): Bar chart for cate gory sales and line chart for hourly sales trends.



• Sales by Discount Type: Bar chart showing sales distribution by discount type.



3.2 Customers Dashboard

The Customers Dashboard focuses on customer analytics with the following features:

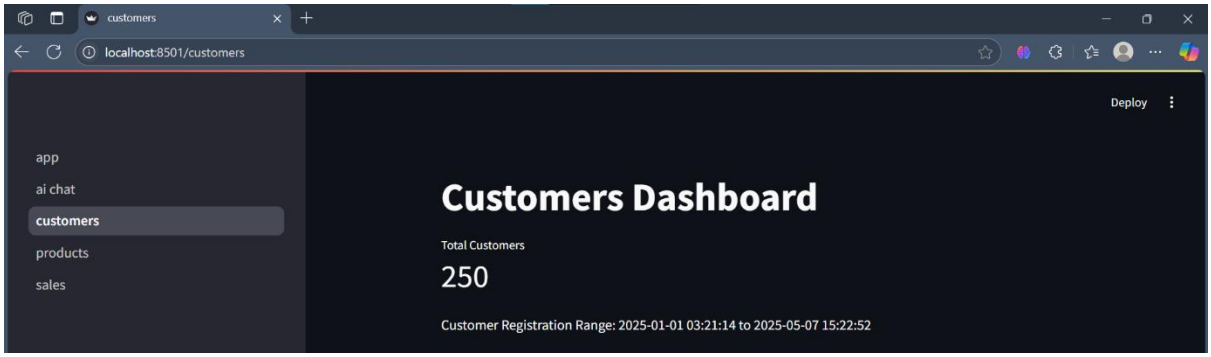
3.2.1 Metrics

• Total Customers: Total number of registered customers.

• 30-Day Customer Retention Rate: Percentage of customers who repurchased within 30 days.

• Average Order Value per Customer: Average revenue per customer order.

• Average Session Duration (Minutes): Average duration of customer sessions.



3.2.2 Data Tables and Charts

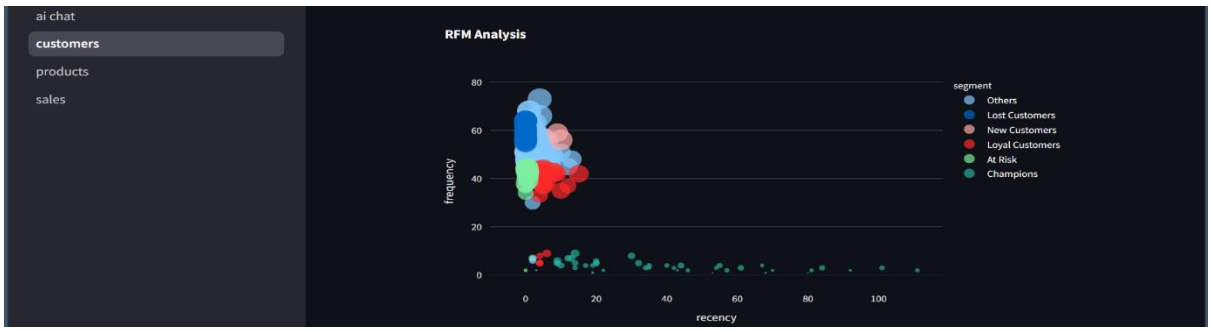
• Top10 Customers by Orders & Sessions: Table of customers with the highest order and session counts.

	customer_id	full_name	total_orders	total_sessions
0	212	Ryan Chang	73	15
1	90	Suzanne Bennett	68	14
2	134	Robyn Reed	68	16
3	116	Zachary Aguirre	66	18
4	189	Mark Spence	65	23
5	55	John Perez	64	32
6	91	Dana Farrell	64	14
7	188	Tiffany Boone	64	17
8	190	Leslie Alvarado	64	47
9	61	Emily Torres	63	44

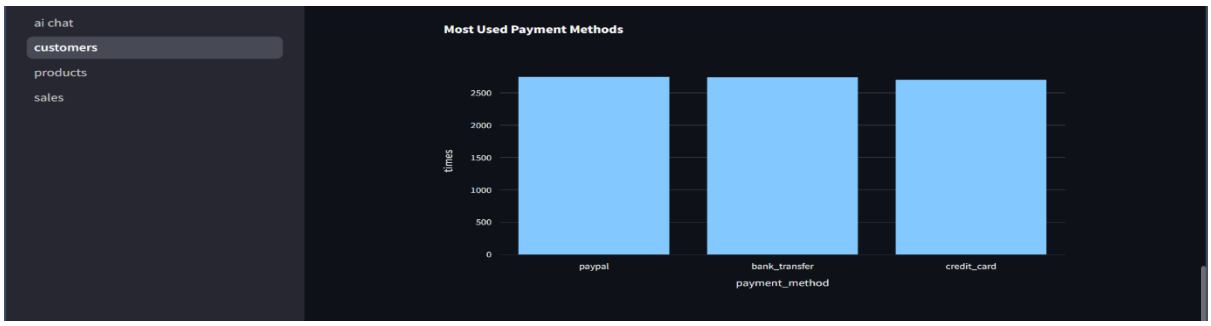
• Top 5 Customers by Customer Lifetime Value (CLV): Table of customers with the highest CLV.

	customer_id	full_name	clv
0	116	Zachary Aguirre	329685.39
1	91	Dana Farrell	308903.76
2	134	Robyn Reed	307115.89
3	212	Ryan Chang	302643.41
4	90	Suzanne Bennett	297327.19

• RFM Analysis: Scatter plot analyzing Recency, Frequency, and Monetary value to segment customers.



• Most Used Payment Methods: Bar chart showing the most popular payment methods.



3.3 Products Dashboard

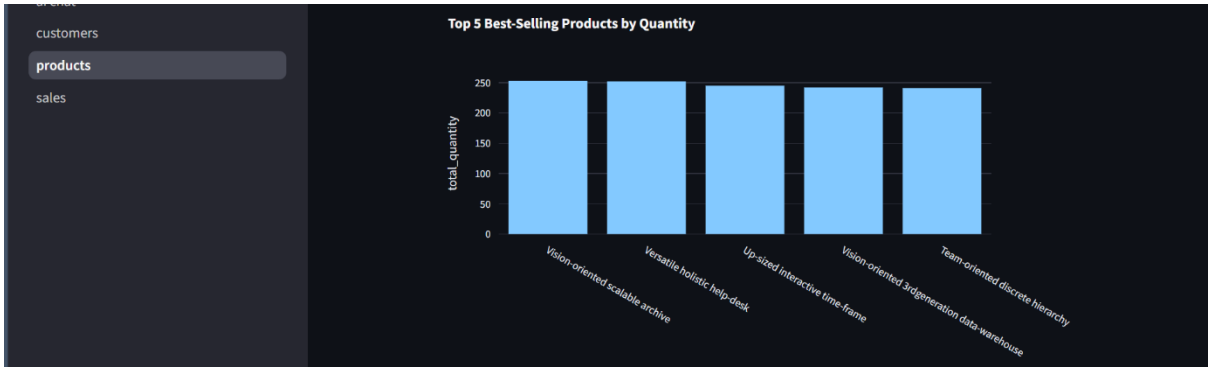
The Products Dashboard provides insights into product performance and inventory with the following features:

3.3.1 Metrics

• Total Products: Total number of products in the catalog.

3.3.2 Data Tables and Charts

• Top 5 Best-Selling Products: Bar chart of the top 5 products by quantity sold.



• Products with Stock Below 20 Units: Table of products with low inventory.

• Top 10 Recommended Products Based on Wishlists: Table of products recommended based on wishlist data.

	wishlist_product	recommended_product	purchase_count
0	Seamless client-driven analyzer	Re-engineered motivating info-mediaries	5
1	Seamless client-driven analyzer	Re-engineered motivating info-mediaries	5
2	Re-engineered motivating info-mediaries	Seamless client-driven analyzer	5
3	Stand-alone zero-defect challenge	Pre-emptive tertiary implementation	4
4	Stand-alone zero-defect challenge	Pre-emptive tertiary implementation	4
5	Stand-alone zero-defect challenge	Pre-emptive tertiary implementation	4
6	Stand-alone zero-defect challenge	Pre-emptive tertiary implementation	4
7	Stand-alone zero-defect challenge	Pre-emptive tertiary implementation	4
8	Ergonomic static hardware	Expanded context-sensitive complexity	4
9	Ergonomic static hardware	Expanded context-sensitive complexity	4

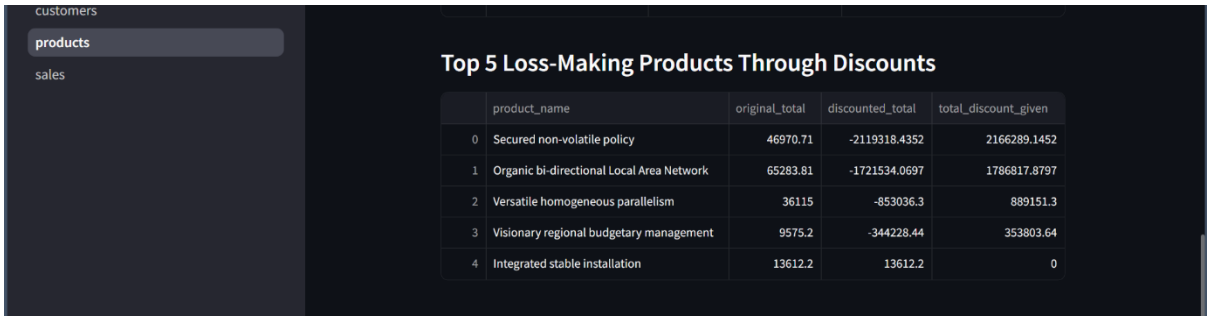
• Top 10 Pairs of Products Commonly Bought Together: Table of frequently co-purchased product pairs.

	product_1	product_2	times_bought_tog
0	Seamless client-driven analyzer	Re-engineered motivating info-mediaries	
1	Adaptive disintermediate matrices	Advanced 6thgeneration adapter	
2	Synchronized upward-trending success	Advanced stable array	
3	Stand-alone secondary neural-net	Advanced zero tolerance hardware	
4	Exclusive client-server challenge	Ameliorated bottom-line middleware	
5	Operative asymmetric benchmark	Automated web-enabled collaboration	
6	Mandatory human-resource moratorium	Cloned hybrid matrices	
7	Triple-buffered homogeneous toolset	Compatible analyzing product	
8	Inverse bandwidth-monitored forecast	Customer-focused disintermediate hierarchy	
9	Multi-lateral well-modulated orchestration	Customer-focused motivating service-desk	

• Top 10 Customers by Total Discount Received: Table of customers with the highest discount amounts.

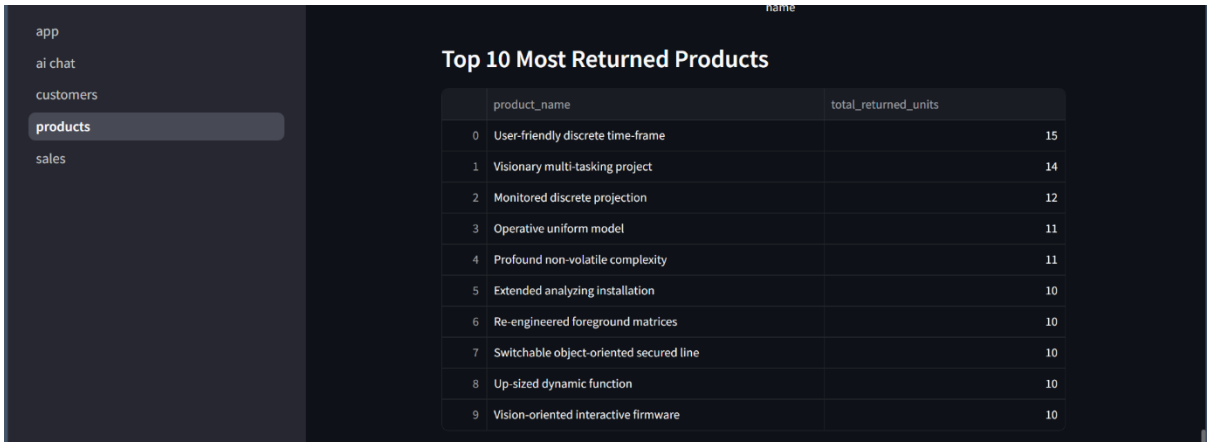
	customer_id	full_name	total_discount_received
0	202	Mason Cruz	241325.964
1	167	Joshua Salas	218803.4612
2	93	Elizabeth Berry	201104.97
3	138	Darin Brown	201104.97
4	179	Erika Humphrey	201104.97
5	211	Phillip Warner	201104.97
6	96	Nicole Underwood	191684.2956
7	127	Joel White	183226.15
8	213	Michael Holland	182741.0834
9	198	Marc Wilson	176376.455

• Top 5 Loss-Making Products Through Discounts: Table of products with the highest discount-related losses.



	product_name	original_total	discounted_total	total_discount_given
0	Secured non-volatile policy	46970.71	-2119318.4352	2166289.1452
1	Organic bi-directional Local Area Network	65283.81	-1721534.0697	1786817.8797
2	Versatile homogeneous parallelism	36115	-853036.3	889151.3
3	Visionary regional budgetary management	9575.2	-344228.44	353803.64
4	Integrated stable installation	13612.2	13612.2	0

• Top 10 Most Returned Products: Table of products with the highest return rates.



	product_name	total_returned_units
0	User-friendly discrete time-frame	15
1	Visionary multi-tasking project	14
2	Monitored discrete projection	12
3	Operative uniform model	11
4	Profound non-volatile complexity	11
5	Extended analyzing installation	10
6	Re-engineered foreground matrices	10
7	Switchable object-oriented secured line	10
8	Up-sized dynamic function	10
9	Vision-oriented interactive firmware	10

3.4 AI Chat

The ai_chat.py script is a Streamlit-based web application that provides an AI-powered chat interface for querying an e-commerce database. It uses the Ollama language model to generate SQL Server queries based on natural language questions, executes them against a SQL Server database, and displays the results in a user-friendly dashboard. The application maintains a chat history to track questions, generated SQL queries, and results, making it suitable for data analysts or business users exploring e-commerce data interactively.

history to track questions, generated SQL queries, and results, making it suitable for data analysts or business users exploring e-commerce data interactively.

Purpose

The script enables users to:

- Ask natural language questions about e-commerce data (e.g., sales, customers, products).
- Automatically generate and execute SQL queries using the Ollama model.
- Visualize query results as tables in a Streamlit interface.
- Review chat history to revisit past queries and results.

Database Schema

The script connects to a SQL Server database (ecommerce_db) with the following schema, where all tables are in the dbo schema:

- **dbo.Categories**: Stores product categories.
    - id: Unique category identifier.
    - name: Category name.
    - description: Category description.
    - parent_id: ID of the parent category (for hierarchical categories).
- **dbo.Products**: Stores product details.
    - id: Unique product identifier.
    - name: Product name.
    - description: Product description.
    - price: Product price.
    - category_id: Foreign key to dbo.Categories.
    - supplier_id: Foreign key to dbo.Suppliers.
    - sku: Stock Keeping Unit identifier.
    - stock_quantity: Current stock level.
- **dbo.Customers**: Stores customer information.

    - id: Unique customer identifier.
    - first_name, last_name: Customer's name.
    - email, phone, address: Contact and location details.
    - registration_date: Date of customer registration.
- **dbo.Orders**: Stores order details.
    - id: Unique order identifier.
    - customer_id: Foreign key to dbo.Customers.
    - order_date: Date of the order.
    - total_amount: Total order amount.
    - status: Order status (e.g., pending, shipped).
- **dbo.OrderDetails**: Stores items in each order.
    - id: Unique order detail identifier.
    - order_id: Foreign key to dbo.Orders.
    - product_id: Foreign key to dbo.Products.
    - quantity: Number of units ordered.
    - unit_price: Price per unit at the time of order.
- **dbo.Payments**: Stores payment details.
    - id: Unique payment identifier.
    - order_id: Foreign key to dbo.Orders.
    - customer_id: Foreign key to dbo.Customers.
    - amount: Payment amount.
    - payment_date: Date of payment.
    - payment_method: Method used (e.g., credit card, PayPal).
    - status: Payment status.
- **dbo.Shipping**: Stores shipping details.
    - id: Unique shipping identifier.

    - order_id: Foreign key to dbo.Orders.
    - shipping_date: Date of shipment.
    - tracking_number: Shipment tracking number.
    - carrier: Shipping carrier.
    - status: Shipping status.
- **dbo.Returns**: Stores return details.
    - id: Unique return identifier.
    - order_id: Foreign key to dbo.Orders.
    - return_date: Date of return.
    - reason: Reason for return.
    - status: Return status.
- **dbo.Reviews**: Stores product reviews.
    - id: Unique review identifier.
    - product_id: Foreign key to dbo.Products.
    - customer_id: Foreign key to dbo.Customers.
    - rating: Numerical rating (e.g., 1-5).
    - comment: Review text.
    - review_date: Date of review.
- **dbo.Suppliers**: Stores supplier information.
    - id: Unique supplier identifier.
    - name, contact_person, email, phone, address: Supplier details.
- **dbo.Discounts**: Stores discount codes.
    - id: Unique discount identifier.
    - code: Discount code.
    - percentage: Discount percentage.
    - start_date, end_date: Validity period.

    - is_active: Whether the discount is active.
- **dbo.Wishlists**: Stores customer wishlists.
    - id: Unique wishlist entry identifier.
    - customer_id: Foreign key to dbo.Customers.
    - product_id: Foreign key to dbo.Products.
    - added_date: Date item was added to wishlist.
- **dbo.InventoryMovements**: Tracks inventory changes.
    - id: Unique movement identifier.
    - product_id: Foreign key to dbo.Products.
    - quantity: Quantity moved.
    - movement_type: Type of movement (e.g., restock, sale).
    - movement_date: Date of movement.
- **dbo.CustomerSessions**: Tracks customer sessions.
    - id: Unique session identifier.
    - customer_id: Foreign key to dbo.Customers.
    - session_start, session_end: Session duration.
    - ip_address: Customer's IP address.

Dependencies

- **Python**: Version 3.8 or higher.
- **Streamlit**: For the web interface (pip install streamlit).
- **Pandas**: For handling query results as DataFrames (pip install pandas).
- **SQLAlchemy**: For database connectivity (pip install sqlalchemy).
- **pyodbc**: For SQL Server connection (pip install pyodbc).
- **ODBC Driver 17 for SQL Server**: Required for SQL Server connectivity.
- **Ollama**: For generating SQL queries using a language model.

- Install Ollama and ensure the llama3.1 model is available.
- Follow instructions at Ollama's official site.

Setup Instructions

1. **Install Dependencies**:

```
pip install streamlit pandas sqlalchemy pyodbc
```

2. **Configure Database Connection**:

- Update the create_engine line in ai_chat.py with your SQL Server details if different:
- ```
  engine = create_engine('mssql+pyodbc://@NADA-AHMED\\SQLEXPRESS/ecommerce_db?driver=ODBC+Driver+17+for+SQL+Server&Trusted_Connec
  ```
  Replace YOUR_SERVER_NAME and YOUR_INSTANCE with your SQL Server details.

3. Set Up Ollama:

- Install Ollama and pull the llama3.1 model:
  ```
  ollama pull llama3.1
  ```
  Ensure the Ollama service is running locally or on a server accessible to the script.

4. Run the Application:

```
streamlit run ai_chat.py
```

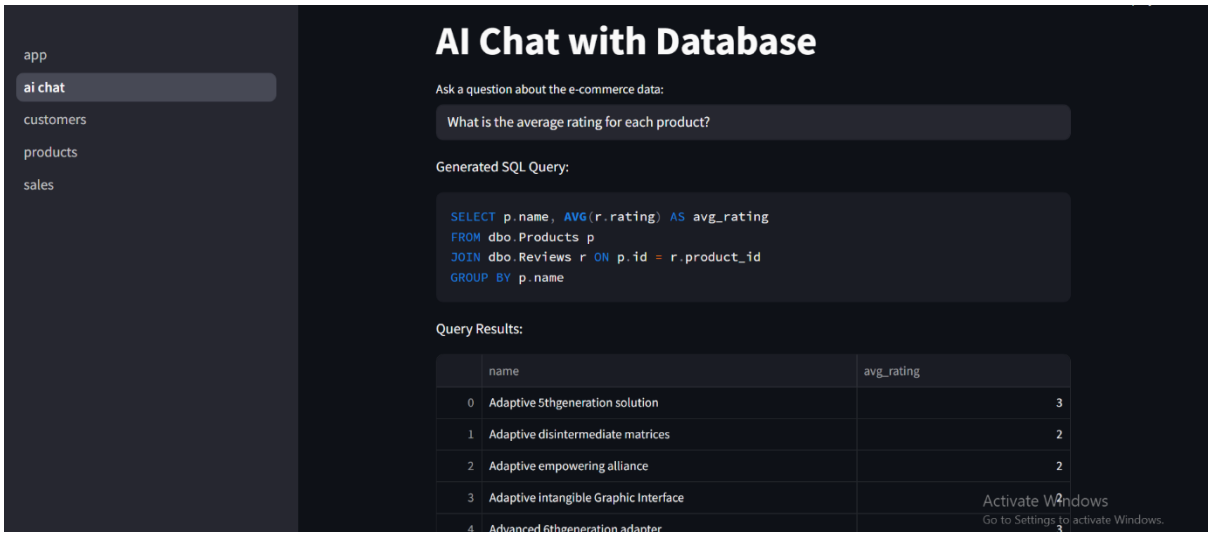
This starts the Streamlit server, typically accessible at http://localhost:8501.

Usage

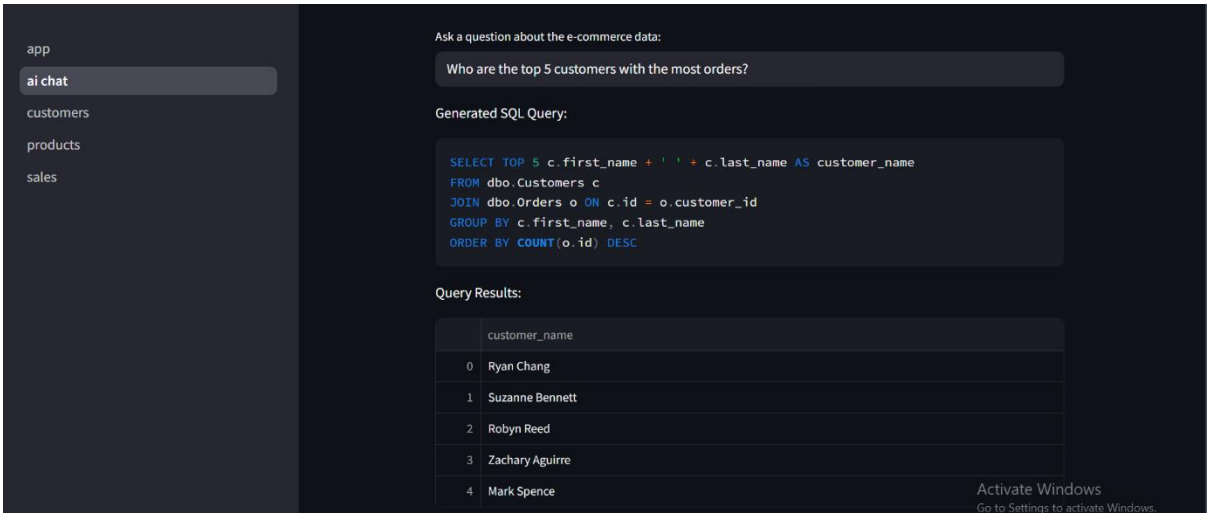
1. **Access the Dashboard**:
   - Open the Streamlit app in a browser (default: http://localhost:8501).
   - The interface displays a title ("AI Chat with Database"), a text input for questions, and a chat history section.
```

2. **Ask Questions**:
    - Enter a natural language question about the e-commerce data (e.g., "What are the top 5 best-selling products by quantity?").
    - The script uses Ollama to generate a SQL query, executes it, and displays the results as a table.
    - Errors (e.g., invalid SQL) are shown if the query fails.
3. **View Chat History**:
    - The chat history section shows all previous questions, their generated SQL queries, and results (or errors).
    - Results are displayed as interactive DataFrames using Streamlit's st.dataframe.
4. **Example Queries**: Below are examples aligned with the dashboard queries from the e-commerce dashboard requirements:

    ➢ **Average product ratings**



    ➢ **Top 5 customers by order count**



Sales Forecasting Project

1. Problem Statement

The goal of this project is to forecast weekly sales based on historical order data. This enables the business to better plan inventory, schedule marketing campaigns, and set realistic sales targets.

---

2. Data Preparation

Dataset: `orders.csv`

Steps:

- Removed unshipped or cancelled orders
- Converted the `created_at` column to datetime format
- Aggregated the data by week using the start of each week (Monday)

Key Libraries:

```
import pandas as pd
```

---

3. Time Series Aggregation

Aggregated the cleaned data by week:

```
df = pd.read_csv("C:/Users/Zerooo-2024/Desktop/data/orders.csv")
df = df[df['status'] == 'shipped']
df['created_at'] = pd.to_datetime(df['created_at'])
df.set_index('created_at', inplace=True)
```

```
df_weekly = df['total'].resample('W-MON').sum().reset_index()
df_weekly.rename(columns={'created_at': 'ds', 'total': 'y'}, inplace=True)
```

---

4. Forecasting using Prophet

Model Used: Facebook Prophet

Forecast Horizon: 8 weeks ahead

```
from prophet import Prophet
model = Prophet()
model.fit(df_weekly)
future = model.make_future_dataframe(periods=8, freq='W')
forecast = model.predict(future)
```

Forecast Outputs:

- `ds`: date
- `yhat`: forecasted sales
- `yhat_lower`: lower bound
- `yhat_upper`: upper bound

4.5. Merging Actual with Forecast

```
merged = df_weekly.merge(forecast[['ds', 'yhat']], on='ds',
how='left')
```

---

5. Visualizations

a. Actual vs Forecasted (Using Plotly)

```
import plotly.graph_objs as go
from plotly.offline import init_notebook_mode, iplot

init_notebook_mode(connected=True)

# رسم البيانات
fig = go.Figure()

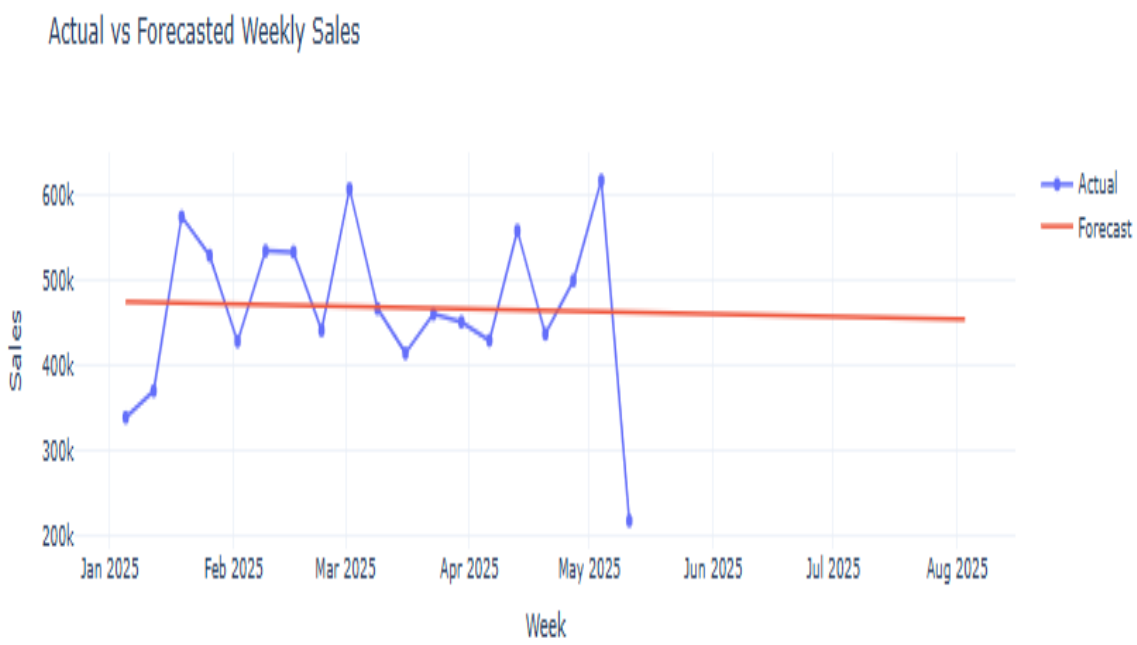
# Actual
fig.add_trace(go.Scatter(x=merged['ds'], y=merged['y'],
mode='lines+markers', name='Actual'))

# Forecast
fig.add_trace(go.Scatter(x=merged['ds'], y=merged['yhat'],
mode='lines', name='Forecast'))
```

```
fig.update_layout(
    title='Actual vs Forecasted Weekly Sales',
    xaxis_title='Week',
    yaxis_title='Sales',
    template='plotly_white'
)

iplot(fig)
```

📎 **Chart: Actual vs Forecasted Weekly Sales**



Suggestions & Fixes:

- The forecast line is almost flat while the actual values are more variable. To improve this:
  - Consider adding holiday effects or promotional data if available.
  - Check if there are sudden drops in data (e.g., May 2025). If it's a data issue, fix the source.
  - Experiment with `seasonality_mode='multiplicative'` if sales have increasing variation.
- Re-train the model frequently with the latest data to adjust the forecast more accurately.
- Validate any unexpected values using raw order logs or source systems.

---

6. Business Recommendations

- **Target Setting**: Use the forecasted values (`yhat`) to set realistic weekly sales targets.
- **Marketing Planning**: Increase ad spending during forecasted low-sales weeks.
- **Inventory Management**: Prepare more inventory during high-forecast weeks.
- **Continuous Monitoring**: Regularly update the model with new sales data to improve forecast accuracy.

---

7. Insights & Observations

- **Unexpected Dip**: A sharp drop in actual sales is observed in May 2025. This could be due to:
  - A reporting issue or data entry error
  - A real business disruption (e.g. holiday, inventory issue, delivery problem)
  - Recommended action: Cross-verify raw data and business logs
- **Flat Forecast**: The forecasted trend is nearly flat. This may suggest:
  - Insufficient seasonality or trend detected
  - The model could benefit from adding holidays or extra regressors
  - Try including external factors (promotions, holidays, campaigns) in future models

---

8. Notes

- Model assumes no holidays or special events unless explicitly added.
- Prophet is chosen for its flexibility with seasonality and trend components.
- Can be expanded to monthly or daily forecasts if needed.
- Plotly was used for interactive visualization with better clarity.

Machine Learning Final Phase

1. Data Preparation

Extracted relevant attributes from multiple SQL tables (e.g., customers, orders, discounts).

Cleaned the dataset: handled missing values, removed duplicates, and adjusted data types.

Engineered features:

over_discounted: top 20% of total discount usage.

inactive: bottom 25% in session count or active days.

Performed basic EDA (correlations, distributions, outliers).

---

2. Modeling & Logic

Trained two Random Forest models:

Model 1: Predict over-discounted customers.

Model 2: Identify inactive customers.

Applied scaling and split data (75% train / 25% test).

Evaluated using classification reports and confusion matrices

Created a discount suggestion function based on behavior.

Integrated logic to decide

If over-discounted → block discounts.

If inactive → offer dynamic discount (5–20%).

Else → no action.

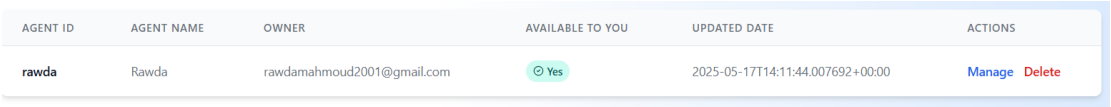
Vanna AI

As part of the integration process, I first **added Vanna AI's service identity to Azure Active Directory** and configured it as an **external user**. This enabled secure, authorized access for the Vanna agent to connect to our SQL Server instance hosted in Azure.

I handled the full setup, training, and optimization of a Vanna AI agent to interact with our SQL Server database. My work included the following:

1. Creating the Agent

I created a new Vanna AI agent from scratch to serve as an intelligent interface for generating SQL queries using natural language.



AGENT ID	AGENT NAME	OWNER	AVAILABLE TO YOU	UPDATED DATE	ACTIONS
rawda	Rawda	rawdamahmoud2001@gmail.com	Yes	2025-05-17T14:11:44.007692+00:00	Manage Delete

2. Connecting to the Database

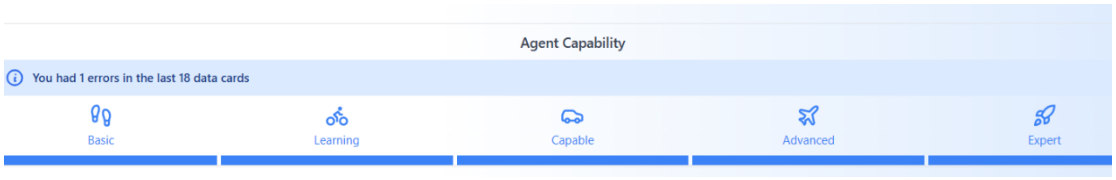
I established a successful connection between the agent and our SQL Server Management Studio (SSMS) database.
This enabled the agent to understand our data structure and start interacting with real data.

3. Training the Agent

Once the connection was established, I trained the agent using the schema and structure of our actual data.
I provided the necessary input to help it understand tables, relationships, and context.

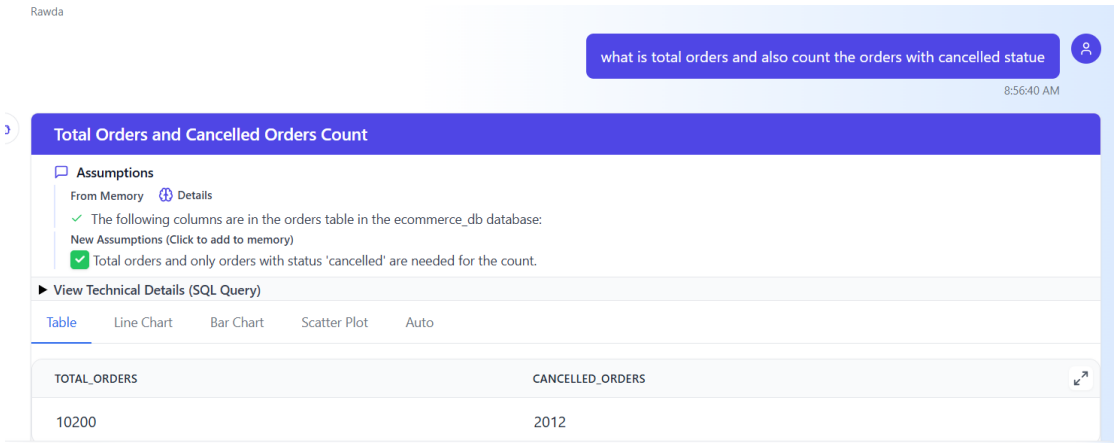
4. Interactive Chat-Based Improvement

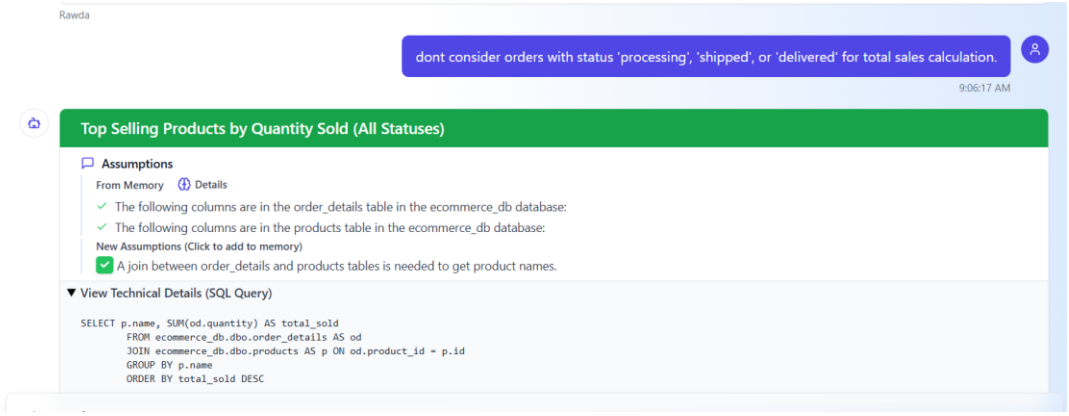
I asked the agent multiple questions using the chat feature and evaluated the SQL queries it generated.
When needed, I adjusted my questions or gave clarifying inputs to guide the agent toward the correct answers.
I kept interacting until the capability bar was fully filled, indicating the agent had reached a high level of understanding and accuracy.



Result

The trained agent is now highly effective at answering natural language questions with accurate SQL queries.
It also provides assumptions or suggestions when necessary to help reach the right answers faster





Action Plan

Project Goal / Mission Statement

The goal of this data analysis project is to identify the reasons behind the decline in e-commerce sales, with a specific focus on the significant drop observed in the month of May.
By analyzing customer behavior, order patterns, and cancellation trends, this project aims to uncover actionable insights that can help the business improve performance and retention.

Step 1: Customer Trend Analysis (Monthly)

To understand the root cause of the sales decline—particularly in May—the first area analyzed was customer trends over the past five months.

Data Summary

Month	New Customers	Returning Customers	Sales ($M)
January	16	359	11.00
February	21	854	10.07
March	18	2,000	10.82
April	13	2,000	10.79
May	5	505	2.26
Total	73	5,500	44.90

Key Insights

1. New Customer Acquisition Collapsed in May

- Only 5 new customers joined in May, down from:
  - 13 in April
  - 18 in March
  - 21 in February
- This is the lowest new customer count by far, suggesting serious acquisition issues.

Recommendation:
Fix New Customer Acquisition

- Reinvest in digital marketing, SEO, social media influencers, or paid ads.
- Audit changes made in May: budget cuts, channel shutdowns, campaign pauses?

---

2. Returning Customers Dropped Sharply

- May saw only 505 returning customers, down from 2,000 in March and April.
- That's a 75% drop in customer retention.

Recommendation:
Re-engage Returning Customers

- Run reactivation email campaigns or SMS reminders.
- Offer loyalty programs or discounts for second purchases.
- Survey past buyers to understand churn reasons.

Step 2: Cancellation Rate Analysis

After analyzing customer trends, the next critical metric reviewed was the order cancellation rate, as it can directly impact both revenue and customer satisfaction.

Data Summary

- Total Orders: 10,000
- Cancelled Orders: 2,000
- Cancellation Rate: 20%

A 20% cancellation rate is significantly higher than industry norms. Most healthy e-commerce operations maintain a cancellation rate under 5%.

Common Causes of High Cancellations

1. Inventory & Stock Management Issues

- Products are marked available online but are out of stock during fulfillment.
- Poor synchronization between website inventory and warehouse systems.

2. Customer-Driven Cancellations

- Customer changes mind post-purchase.
- Found better deals elsewhere.
- Unattractive shipping fees or long delivery estimates.

3. Poor User Experience (UX)

- Double orders due to interface glitches.
- Confusing navigation or unclear order confirmation messages.
- Sluggish site performance during high traffic.

Recommended Actions

1. Capture & Classify Cancellation Reasons

- Add a "Reason for Cancellation" dropdown for both customers and support staff.
- Tag and categorize cancellations into buckets:
  Inventory, Shipping, Fraud, UX, Customer Choice, etc.

2. Monitor Patterns

- Track cancellation reasons over time to identify trends.
- Segment by product, campaign, payment method, or geography.

Step 3: Shipping Delay Analysis

Another potential cause of customer dissatisfaction and order cancellations is shipping delay — the gap between when an order is placed and when it is shipped.

What Was Analyzed

- The time difference between the Order Date and Shipping Date.
- The goal was to determine whether delays in shipping could be contributing to poor customer retention or cancellation rates.

Key Finding

- Currently, orders are being shipped on the same day they are placed.
- There is no significant delay between the order and shipping timestamps.